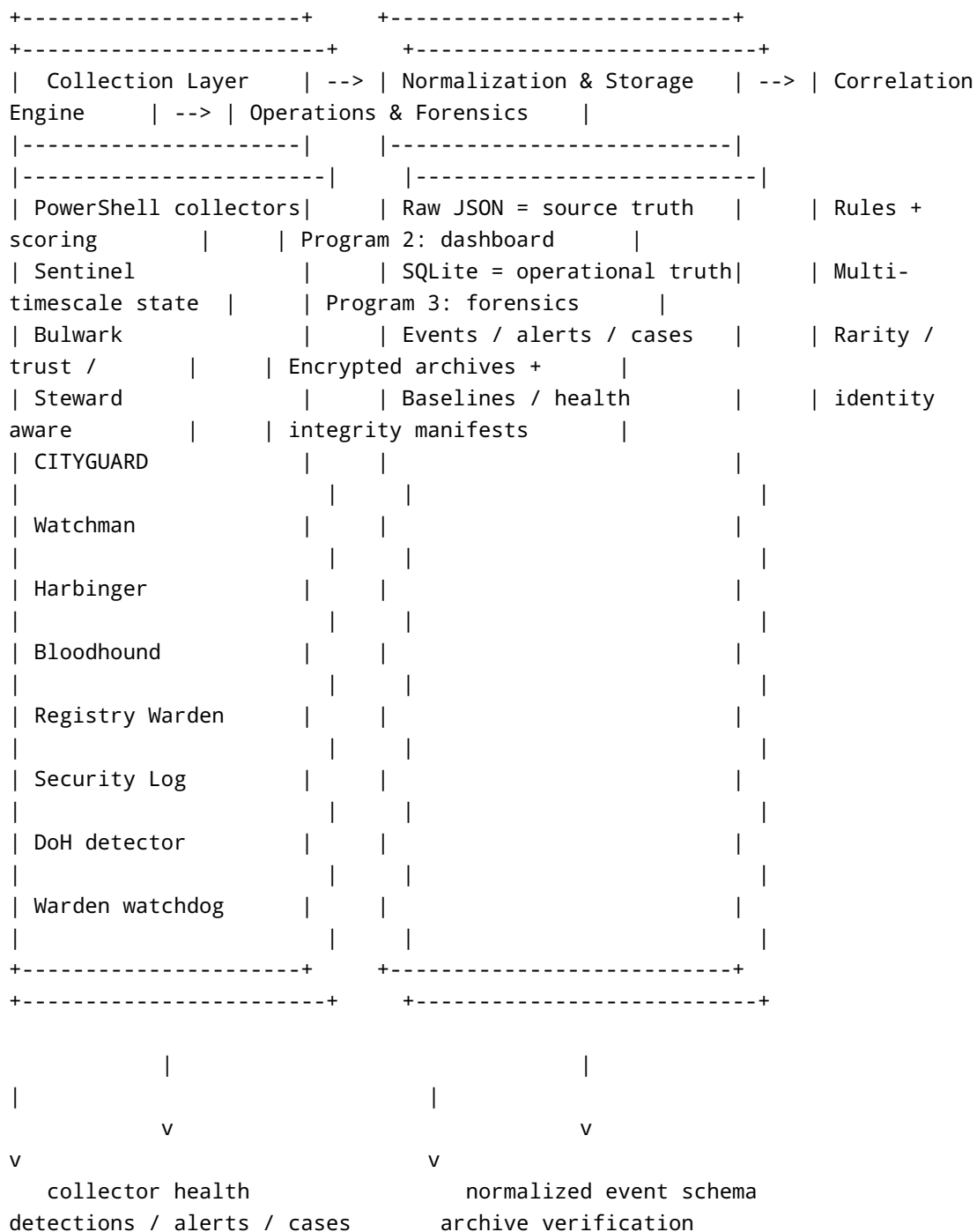


Home SOC Suite — Architecture Diagram and Correlation Engine Design

Target architecture



Design principles

1. Raw JSON remains source truth; SQLite is operational truth.
2. Missing data is itself a signal.
3. Every alert must stay explainable with evidence, score factors, and confidence.
4. Program 3 reconstructs narratives from archives, not just file searches.
5. Baselines, archives, and schemas need versioning to resist rollback and replay.

Correlation engine design

Program 1 mission

Program 1 is the operational brain of the suite. It should ingest raw telemetry, normalize it, preserve trust metadata, run explainable rules, accumulate weak signals over time, and promote only evidence-backed detections into alerts and cases.

Core responsibilities

- Ingest raw JSON events from every collector without mutating the original source record.
- Normalize events into a common schema inside SQLite so queries, scoring, and dashboard reads stay consistent.
- Run a hybrid detection model: deterministic rules for clear chains and risk scoring for weak, cumulative signals.
- Maintain state across short, operational, and campaign windows so low-and-slow behavior can still be linked.
- Create alerts only when the engine can explain what contributed to the decision and how confident it is.

Required object boundaries

- **Raw event:** exact collector output, preserved as originally observed.
- **Normalized event:** mapped event with common fields for correlation and queries.
- **Detection:** a rule or score result that says something may matter.
- **Alert:** a user-visible item worth triage.
- **Case:** a grouped investigation unit containing related alerts, detections, and evidence.
- **Evidence artifact:** archive bundle, manifest, hashes, reports, and later forensic imports.

Processing pipeline

1. Collector write

Input arrives from PowerShell collectors.

Minimum fields: - collector_name - observed_at - schema_version - raw_json - raw_hash

Watch for: - silent collectors - malformed output - timestamp drift

2. Normalization

Raw events become normalized events in SQLite.

Minimum fields: - event_type - host - user - session_type - trust_level - parser_confidence - ingest_status - ingested_at

Watch for: - parser drift - dropped fields - duplicate event IDs

3. Correlation

The engine builds detections using time windows and entity grouping.

Minimum fields: - rule_id - window_type - matched_entities - score_factors - confidence - evidence_refs

Watch for: - time desynchronization - replay - out-of-order arrival - missing context

4. Promotion

Detections become alerts or attach to cases.

Minimum fields: - severity - status - explanation - case_id - created_at

Watch for: - false certainty - duplicate cases - noisy promotion logic

5. Archive

Hot data ages out into archive bundles.

Minimum fields: - archive_id - time_range - manifest_hash - previous_hash - verification_status

Watch for: - rollback - tampering - schema incompatibility

Recommended SQLite tables

- **raw_events**: immutable ingest record
- **events**: normalized operational events
- **detections**: intermediate engine output
- **alerts**: dashboard-facing objects
- **cases**: grouped investigation units
- **collector_health**: heartbeat and expected-rate model
- **baselines**: approved normal behavior with versioning
- **archives**: evidence catalog and verification state

Rule engine and scoring model

Hybrid design

Use both: - **Deterministic rules** for clear attack chains - **Risk scoring** for weak signals that become meaningful over time

Deterministic rule examples

- persistence creation followed by process execution and outbound connection

- process start followed by unusual DNS and rare destination traffic
- collector silence after a reboot or login event

Score factor categories

- **Rarity:** first-seen destination, process pair, path, signer mismatch, unusual country
- **Context:** user role, elevation state, session type, parent-child chain, execution time
- **Trust:** collector health, parser confidence, missing corroboration, source reliability
- **History:** previous sightings, baseline state, recurrence over days, unresolved detections
- **Consistency:** whether multiple collectors agree on the same narrative

Alert promotion logic

- **Hard-trigger alert:** deterministic rule matches a high-confidence malicious chain
- **Score-trigger alert:** cumulative score crosses threshold with enough evidence
- **Visibility alert:** telemetry disappears, health drops, or ingest quality degrades
- **Case merge:** new detection attaches to existing case when timing/entity overlap is strong

Dashboard guardrails

Program 2 should show: - freshness - collector health - ingest lag - confidence - explanation path - evidence links - unknown activity as reviewable inventory

Program 3 guardrails

Program 3 should: - verify manifests before archive import - reconstruct timelines using observed time and ingest time - preserve schema and collector versions per archive - focus on narrative reconstruction: what happened, in what order, with what confidence

Pre-coding checklist

- Lock the raw JSON vs SQLite source-of-truth split
- Define event, detection, alert, case, and archive boundaries
- Version schemas, baselines, archives, and collectors
- Implement collector heartbeat and expected-rate monitoring
- Reserve fields for trust, confidence, and identity context
- Adopt the hybrid rules + scoring approach from the beginning
- Define the archive manifest format before coding Program 3

Bottom line

The suite gets stronger when the Python side models uncertainty, trust, and timeline integrity directly in the architecture. More collectors help, but these design choices are what keep the finished system from becoming a noisy monitor instead of a real detection platform.